Lab 3: ML Workloads Part III (Rooflines + NVIDIA-SMI)

CS2470 Advanced Computer Architecture, Fall 2026 · @noa · Oct 23, 2026

We're trying to figure out 2 questions on serving GPUs for AI workloads:

1. What limits how fast a piece of GPU code runs: the math hardware, or the memory feeding it?
2. How much power and energy does running a language model actually use?

Background (mostly for me to make sure I understood)

What a GPU is doing

A GPU has 2 resources that matter here:

- **Compute:** Small execution units (CUDA cores and Tensor Cores) that do multiply-adds. Speed measured in **FLOP/s** (floating-point operations per second). 1 TFLOP/s = 10^12 FLOP/s.
- **Memory bandwidth:** How fast data moves between the GPU's main memory (DRAM, the "24 GB" on the spec sheet) and those execution units. Measured in **GB/s**.

Every computation needs both, because data has to arrive before it can be multiplied. Whichever runs out first sets the speed. The L4 GPU in this lab can do 121 TFLOP/s of fp16 math, but its memory only delivers 300 GB/s.

Arithmetic intensity

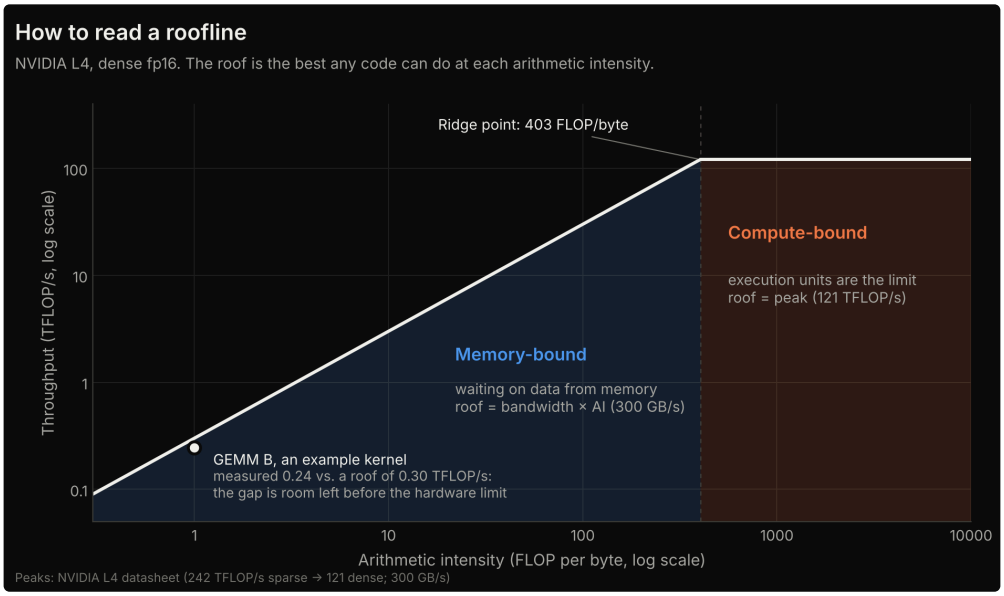
Arithmetic intensity (AI) = FLOPs performed / bytes moved from memory. It measures how much math a program does for each byte it reads.

The roofline model

A roofline plot puts arithmetic intensity on the x-axis and achieved performance (FLOP/s) on the y-axis. The "roof" is the best performance the hardware could reach at each AI.

Roofline represents the hardware (i.e 1 GPU). Drawn from 2 numbers about the chip: peak compute and memory bandwidth. A point on roofline is 1 kernel (1 piece of code) actually

run on that hardware. Whole plot is a map of everything that could run on that chip .



- **Left side (sloped roof):** low AI. Performance is capped at bandwidth × AI. These programs are **memory-bound**, so more execution units wouldn't help.
- **Right side (flat roof):** high AI. Performance is capped at peak FLOP/s. These programs are **compute-bound**.
- **Ridge point:** where the 2 meet, at AI = peak FLOP/s / bandwidth. For the L4 that is 121 TFLOP/s / 300 GB/s ≈ 403 FLOP/Byte.
- Blue dot is an example kernel: Here we took the GEMM B from Exercise II. Since it's far left of the "ridge" point at 403, it's memory-bound. Memory limits it. The point on the red line directly above is the best it could possibly do - the fastest the kernel could run if the GPU's memory delivred data at its full rated speed with 0 waste. Since GEMM B is at a performance of 0.24 TFLOPS/s, it's ~80% of ceiling. Why?
  - Memory never runs at its full rated speed in practice
  - Every kernel launched has fixed stratup overhead (matters especially for short kernel like this one).
- Gap between the dot and roof is the difference between what it achieved and best possible. For this kernel, gap is small so GEMM B is close to as fast as this GPU allows. To go faster, you need to change the kernel to move along the x-axis and do more math per byte (batching) or reading fewer bytes.
- To place a real program on the plot, you compute its AI by hand and measure its runtime. The vertical gap between the dot and the roof is performance left on the table. The horizontal position tells you which resource to blame.

Where the L4 numbers come from

Both come from NVIDIA's L4 spec page. It lists FP16 Tensor Core at **242 TFLOPS**, with the footnote "Shown with sparsity. Specifications are one-half lower without sparsity."

Sparsity skips half the weights when they're zero in a specific pattern. Our matrices are ordinary dense ones, so the real peak is 242 / 2 = **121 TFLOP/s**. Memory bandwidth is listed directly as **300 GB/s**.

The ridge falls where the sloped roof reaches the flat one:

```
bandwidth × AI = peak FLOP/s
```

```
AI = peak FLOP/s ÷ bandwidth
   = (121 × 10¹² FLOP/s) ÷ (300 × 10⁹ B/s)
   ≈ 403 FLOP/B
```

The "per second" cancels, leaving FLOP per byte. These are spec-sheet maximums.

GEMMs and convolutions

- **GEMM** (general matrix multiply) is C = A × B. Multiplying an M×K matrix by a K×N matrix takes 2·M·N·K FLOPs, since each output element needs K multiplies and K adds. Almost all the math in a transformer language model is GEMMs: each layer multiplies the token activations by large weight matrices.
- **Convolution** is the core operation of image models (CNNs). It slides a small filter over an image. GPUs run it as a GEMM in disguise ("implicit GEMM").

Model size: what "8B parameters in fp16" means

- **8B parameters** = the model is about 8 billion numbers (weights) learned during training.
- **fp16** = each number is a 16-bit float, **2 bytes**. Other formats: fp32 (4 bytes), int8 or fp8 (1 byte).
- 8 billion × 2 bytes ≈ **16 GB** of weights the GPU has to hold, and read.

Prefill vs decode (how LLMs generate text)

1. **Prefill:** the model reads the whole prompt at once. With a 1,000-token prompt, every weight matrix is multiplied against 1,000 rows at once. Each weight loaded from memory gets reused 1,000 times, so AI is high and prefill is **compute-bound**.
2. **Decode:** the model then generates 1 token at a time. Each step multiplies every weight matrix against a single row, so each weight is loaded and used once. AI is about 1 and decode is **memory-bound**. For an 8B model in fp16, every token streams ~16 GB of weights. At 300 GB/s that caps the L4 at ~53 ms per token, about 19 tokens/s.

Batching means generating for many prompts at once. During decode at batch 100, each weight read serves 100 rows instead of 1, which raises AI about 100x. Exercise IV measures the effect directly.

Power vs energy

- **Power** (watts, W) is the rate of energy use at a given moment. nvidia-smi reports it.
- **Energy** (joules, J) is the total over time: energy = power × time. A GPU drawing 70 W for 10 s uses 700 J.
- **TDP / power limit** is the maximum power the GPU may draw: 72 W for the L4, small for a data-center GPU (an H100 is 700 W). If a workload would exceed it, the GPU lowers its clock speed to stay under, which makes it slower.
- **P-states** are power modes. P0 is full speed, P8 is deep idle with clocks parked.

For efficiency, the useful number is usually **energy per unit of work** (here, joules per generated token), not raw power.

How the lab is run

We run the lab on slurm.

- **PyTorch Profiler** records how long each GPU operation takes.
- **nvidia-smi** is NVIDIA's command-line tool for reading GPU power, temperature, memory and running processes.

Part I: Constructing a Roofline

The L4's ridge point is 403 FLOP/Byte, much higher than the A100 or H100, because it has very little bandwidth for its compute. The higher the ridge point, the more kernels fall to its left and are memory-bound on that chip. Vs an A100, the L4 has about 1/3 the compute (121 vs 312 TFLOP/s) but only about 1/7 the bandwidth (300 vs 2,039 GB/s)

Q1. Fill in the hardware platform data: look up the peak FLOPS and memory bandwidth of an L4 GPU.

Step 1, hardware data. I used the dense FP16 Tensor Core numbers from NVIDIA's datasheets (L4, A100, H100), without the 2:4 sparsity speedup, since our kernels are dense. Each datasheet shares the sparse figure, so the dense peak is half of it: L4 242 → 121, A100 624 → 312, H100 1,979 → 989 TFLOP/s.

GPU	Peak FP16, dense (TFLOP/s)	Memory BW (GB/s)	Ridge point (FLOP/B)
L4	121	300	**403.3**
A100 SXM 80GB	312	2,039	**153.0**
H100 SXM	989	3,350	**295.2**

```
platforms = [
    {
        'name': 'NVIDIA L4',
        'peak_flops': 121e12,      # 121 TFLOP/s, dense FP16 Tensor Core
        'peak_memory_bw': 300e9,   # 300 GB/s
        'color': 'tab:red'
    }
]
```

Units are FLOP/s and bytes/s, so 121 TFLOP/s = 121e12 and 300 GB/s = 300e9. Exercise I adds the A100 and H100 as 2 more entries in this list.

Q2. Calculate the intersection point of the roofline, where the diagonal and flat parts connect.

Step 2, intersection point.

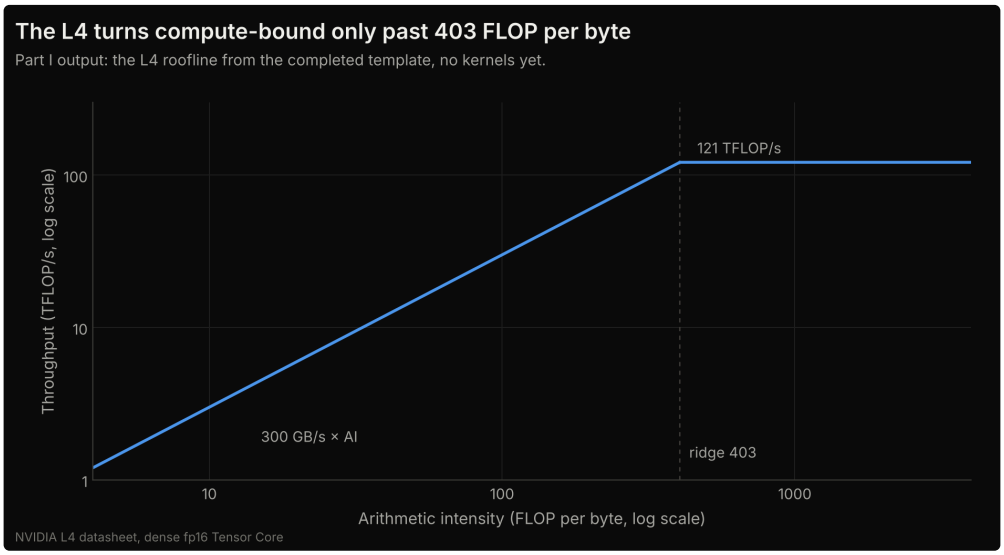
```
x_intersection = peak_flops / peak_memory_bw
intersections.append({'name': name, 'x_intersection': x_intersection, ...})
```

Q3. Plot the actual curves of the roofline.

Step 3, plot the curves. A sloped line from min_x to the ridge, then a flat line from the ridge to max_x:

```
x_mem = np.geomspace(min_x, x_intersection, 200)
ax.plot(x_mem, peak_memory_bw * x_mem, color=color, label=name)          #
memory roof
ax.plot([x_intersection, max_x], [peak_flops, peak_flops], color=color)  #
compute roof
```

Running the completed template gives the L4 roofline:



on an L4, a program has to do about 400 FLOPs for every byte it reads before the execution units become the bottleneck. Anything less, and the GPU spends its time

waiting on memory.

Part II: Adding Kernels to the Roofline

Q4. Enable profiling of the GEMM kernel with PyTorch Profiler, calculate its execution time, and plot the kernel on the roofline.

The 4096^3 GEMM runs at 76.8 TFLOP/s, 63% of the L4's 121 TFLOP/s peak, and is compute-bound.

A **kernel** is 1 function that runs on the GPU, such as 1 matrix multiply. To put a kernel on the roofline it needs 2 numbers: its AI (x-axis) and its achieved FLOP/s (y-axis). AI = FLOPs / bytes, and throughput = FLOPs / time. FLOPs and bytes can be worked out on paper from the matrix sizes. Time has to be measured.

For a GEMM (M×K) @ (K×N):

```
FLOPs = 2 · M · N · K
Bytes = 2 · (M·K + K·N + M·N)
```

The 2 in FLOPs is 1 multiply plus 1 add per MAC. The 2 in bytes is fp16's size, counting A and B read once and C written once. "4096^3" means M = K = N = 4096, i.e 2 matrices, each 4096×4096. That gives 1.374e11 FLOPs and 1.007e8 bytes, so **AI = 1,365 FLOP/B**.

Profiling.

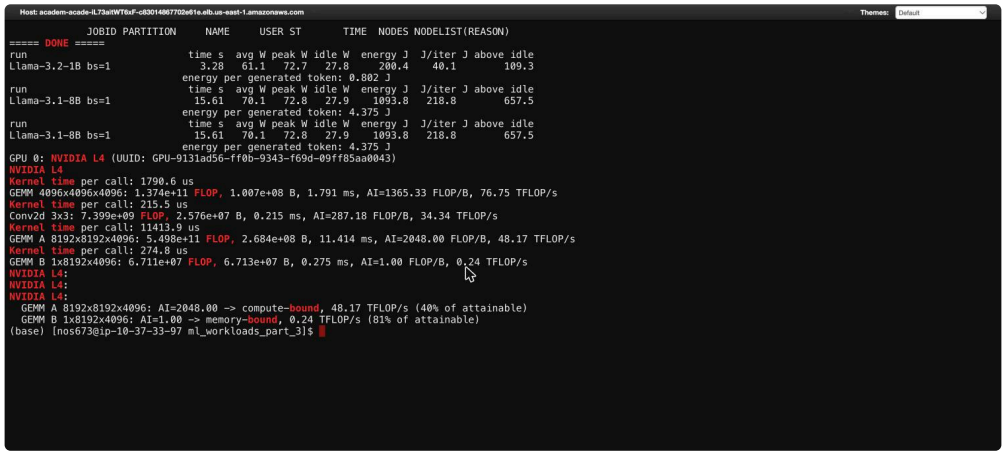
1. Wrap the GEMM in PyTorch Profiler: records exactly how long each kernel ran *on the GPU*. A normal Python timer would also count Python overhead and launch delays, not just the GPU's work.
2. Run 10 warmup iterations: The first few runs are slow for one-time reasons: libraries load, PyTorch tries a few kernels to find the fastest one, and the GPU's clocks speed up from idle. Timing those would make the GEMM look slower than it really is.
3. Profiled 20 and divided the GPU kernel time by 20. Get the average cross several runs.

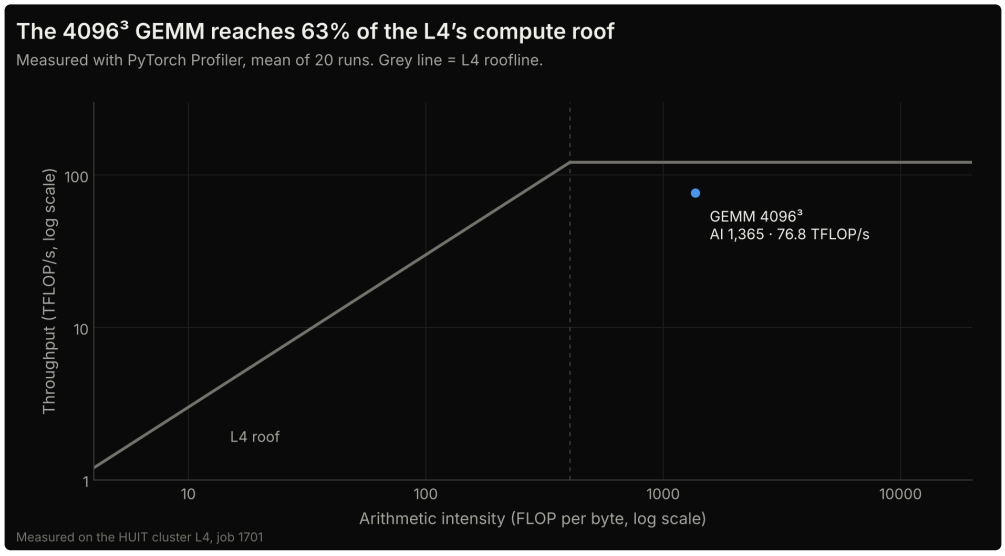
```
# Run 10 untimed so the measurement only sees steady-state speed.
with torch.inference_mode():
    for _ in range(10):
        A @ B
torch.cuda.synchronize()   # make sure warmup is fully done before timing

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA]) as
prof:
    for _ in range(20):
        A @ B
    # GPU work launches asynchronously: without this, the profiler could
    # stop while kernels are still running and miss part of their time.
    torch.cuda.synchronize()
```

```
# Sum only GPU (CUDA) events. CPU-side ops like aten::mm also carry the
# time of the kernels they launch, so summing every row double counts.
total_us = sum(e.self_device_time_total for e in prof.events()
               if e.device_type == DeviceType.CUDA)
per_call_s = total_us / 20 / 1e6   # average time of 1 GEMM, in seconds
```

Result: 1.791 ms per call, so **76.8 TFLOP/s**. The kernel PyTorch picked is ampere_fp16_s1688gemm_fp16_128x128_ldg8_f2f_stages_3...





Exercise I: Plotting Rooflines

The same convolution is memory-bound on the L4 but compute-bound on the A100 and H100: whether a kernel is "memory-bound" depends on the hardware, not just the kernel.

Q5. Add additional rooflines to represent A100 and H100 GPUs.

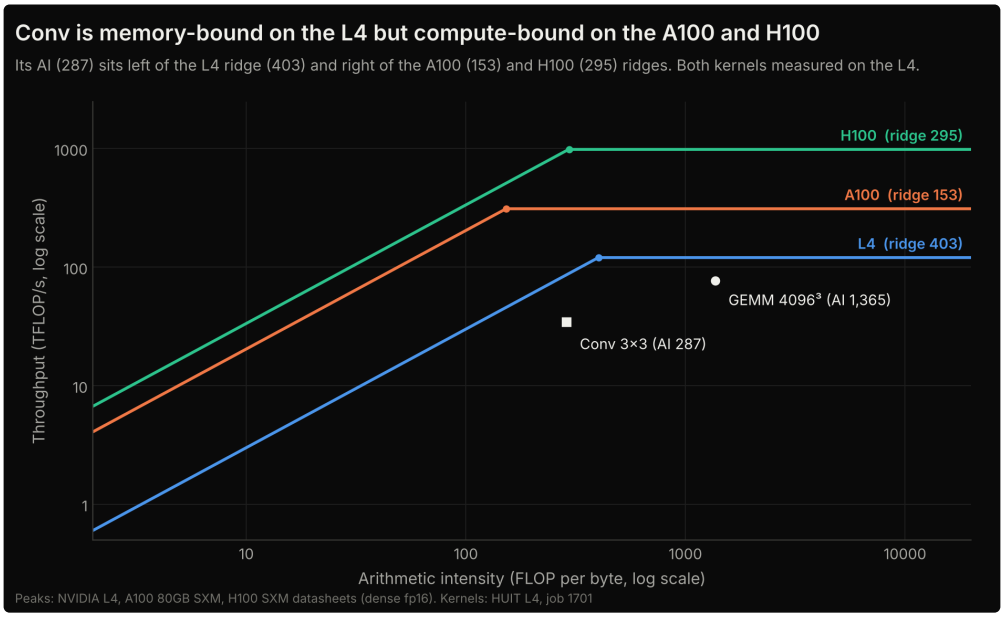
1. A100 and H100 rooflines are 2 more entries in platforms (the Python list of GPU specs from Q1), using the specs in the Part I table.

Q6. Add a convolution kernel to the roofline plot, in addition to the GEMM kernel.

2. Convolution kernel. I used a ResNet-50 conv2_x layer: batch 32, 64→64 channels, 56×56 image, 3×3 filter, stride 1, padding 1.

```
FLOPs = 2 · N · K · H_out · W_out · C · R · S = 7.399 × 10⁹
Bytes = 2.576 × 10⁷
AI    = 287 FLOP/B
```

Bytes = 2 × (input + weights + output), in fp16. **Result:** 0.215 ms, **34.3 TFLOP/s**.



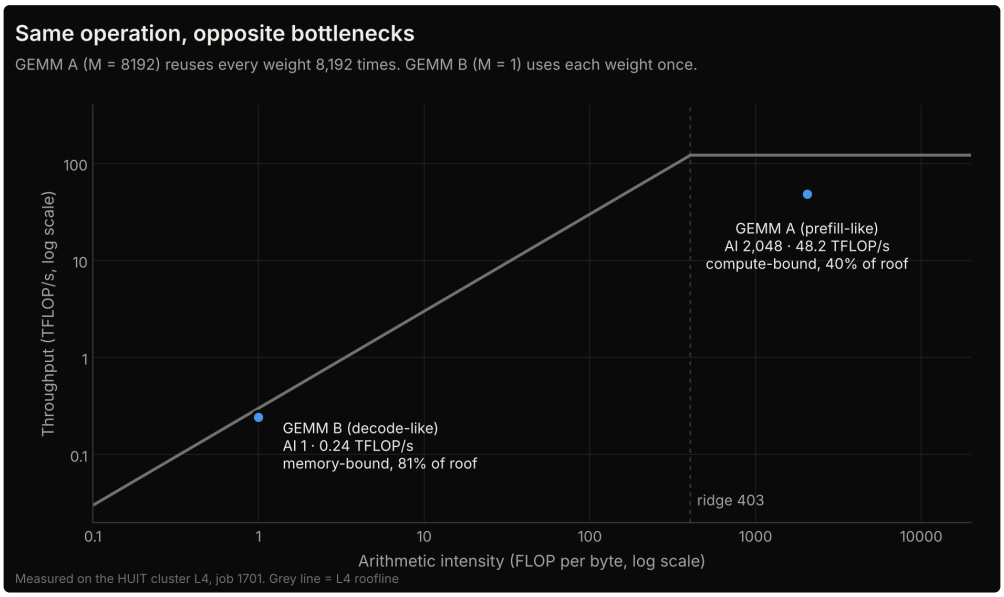
- **GEMM (AI 1,365)** is right of every ridge, so it is compute-bound on all 3 GPUs. On the L4 it reaches 63% of the compute roof.
- **Conv (AI 287)** sits left of the L4 ridge (403) but right of the A100 (153) and H100 (295) ridges.
- **Conv efficiency** is only 40% of its L4 roof (86 TFLOP/s attainable at AI 287). Part of that is layout conversion. The actual convolution kernel (sm86_xmma_fprop_implicit_gemm_...nhwc) is only 52% of the GPU time. The rest converts the input from PyTorch's default NCHW layout (all of channel 1, then all of channel 2) to the NHWC layout the kernel wants (all channels of pixel 1, then pixel 2). Using channels_last tensors would remove that.

Exercise II: Compute-Bound vs Memory-Bound Kernels

Q7. Plot GEMM A (M=8192, K=8192, N=4096) and GEMM B (M=1, K=8192, N=4096) on the L4 roofline. Is each kernel compute-bound or memory-bound?

GEMM A is compute-bound and matches prefill. GEMM B is memory-bound and matches decode, running at 81% of the L4's memory bandwidth.

Kernel	Shape (M, K, N)	AI (FLOP/B)	Time (ms)	Throughput (TFLOP/s)	Bound	% of attainable
GEMM A	8192, 8192, 4096	2,048	11.414	48.2	**compute**	40%
GEMM B	1, 8192, 4096	1.0	0.275	0.24	**memory**	81%



TLDR: GEMM A is a large matrix-matrix multiply with high data reuse: every weight loaded is used across 8,192 rows, so the execution units are the bottleneck. GEMM B is effectively a matrix-vector multiply (GEMV): 1 row against the full weight matrix, so every weight loaded is used once and the GPU mostly waits on memory.

GEMM A (AI 2,048) sits about 5x to the right of the ridge (2,048 vs 403 on the x-axis). Each weight is loaded from memory once and then used in 8,192 multiply-adds, 1 for each of the 8,192 input rows. That reuse is what makes the GPU do a lot of math per byte it fetches.

GEMM B (AI 1.0) sits about 400x below the ridge. With M = 1, each weight is read once and used for 1 multiply-add. Almost all the bytes are the 8192×4096 weight matrix (64 MiB). The kernel moves 67 MB in 0.275 ms, which is **244 GB/s, or 81% of the L4's 300 GB/s**. It is close to what the hardware allows. The only ways to speed it up are to move fewer bytes (quantize the weights) or batch more rows.

Q8. Which phase of LLM inference does each kernel correlate with?
Mapping to LLM inference:

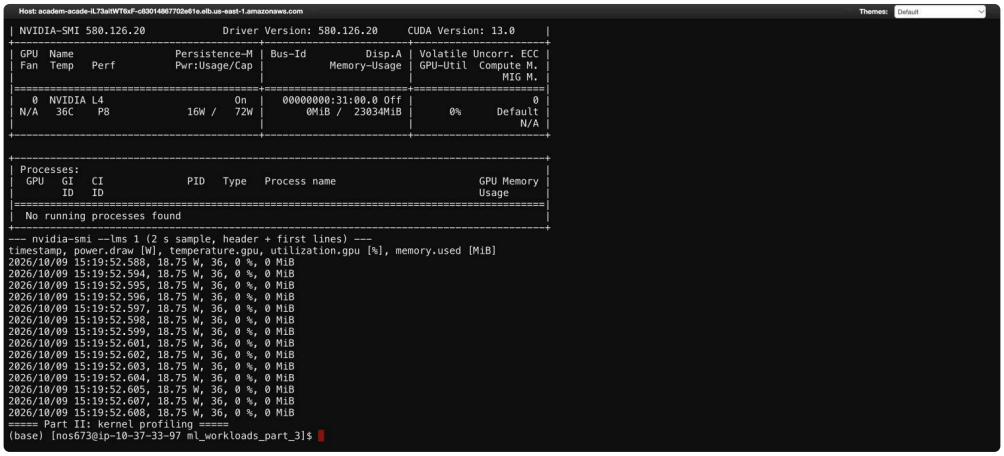
- **GEMM A = prefill.** The whole prompt (8,192 tokens) is processed at once, so M = number of tokens and every weight is reused across all of them.
- **GEMM B = decode.** Each step generates 1 token, so M = 1 and the full weight matrix streams from DRAM for a single row of math. This is why decode is memory-bound and why batching helps so much.

GEMM A only reaches 40% of peak while the smaller 4096^3 GEMM reaches 63%. I believe this is the 72 W power limit. GEMM A runs 11 ms per call for 20 calls back to back, long enough for the L4 to drop its clocks to stay under the cap (Part IV shows the GPU pinned at 72 W under any sustained load). I didn't log clocks during this run, so treat this as the likely cause rather than a measured one.

Part III: Basic Measurements with NVIDIA-SMI

Q9. Run nvidia-smi and note the information it shows: power usage, total power (TDP), temperature, memory used and running processes.

The idle L4 draws 16 W of its 72 W cap, sits at 36 C in its deepest idle state (P8), and has no processes or memory in use.



Field	Value	Meaning
Power usage	**16 W**	Current draw. Nothing is running
Power cap	**72 W**	The power limit (TDP for the L4)
Temperature	**36 C**	GPU core temperature. Cool, since the GPU is idle
Perf state	**P8**	Deepest idle state. Clocks are parked
Memory used	**0 MiB / 23,034 MiB**	GPU memory in use out of the 24 GB total. Nothing is loaded yet
Processes	**none**	Programs currently using the GPU. Empty, so the GPU is free

nvidia-smi --lms 1 (and the --query-gpu ... -lms 1 form used in Part IV) prints a row about every 1 ms. The power value only updates about every 50 ms though, so most rows repeat the last reading: 1 ms logging doesn't give 1 ms power resolution. In the sample at the bottom of the screenshot, power reads 18.75 W on every row.

Part IV: Capturing Power During Workload Execution

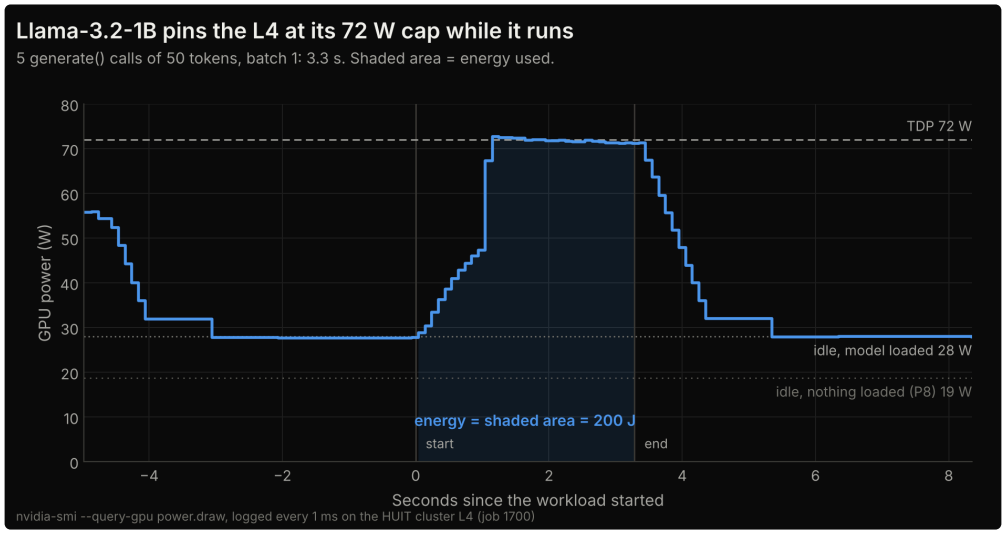
Q10. Capture power consumption while a workload runs, using nvidia-smi in a subprocess, and examine the resulting CSV.

Llama-3.2-1B used 200 J for 5 generate() calls (3.28 s), about 0.80 J per generated token, with the GPU pinned at its 72 W cap while it ran.

The idea: start nvidia-smi logging power to a file in the background, run the model, then stop the logger and line up the readings with when the model was running. A **subprocess** is a second program launched from Python that runs alongside it.

profile_power_workload.py runs: launch nvidia-smi --query-gpu=timestamp,power.draw,... -lms 1 > log.csv as a subprocess, sleep 5 s, run 5 × generate(max_new_tokens=50), sleep 5 s, stop the subprocess. Changes:

1. 1 warmup generate() **before** logging starts, so CUDA setup and kernel tuning aren't in the measurement
2. The workload's start and end timestamps are saved to a JSON next to the CSV, so the energy sum covers exactly the workload window
3. min_new_tokens=50 and greedy decoding, so every run generates exactly 50 tokens per prompt and runs stay comparable



Time (s)	Avg power (W)	Peak (W)	Energy (J)	Per generate() (J)	Per token (J)
3.28	61.1	72.7	**200**	40.1	0.80

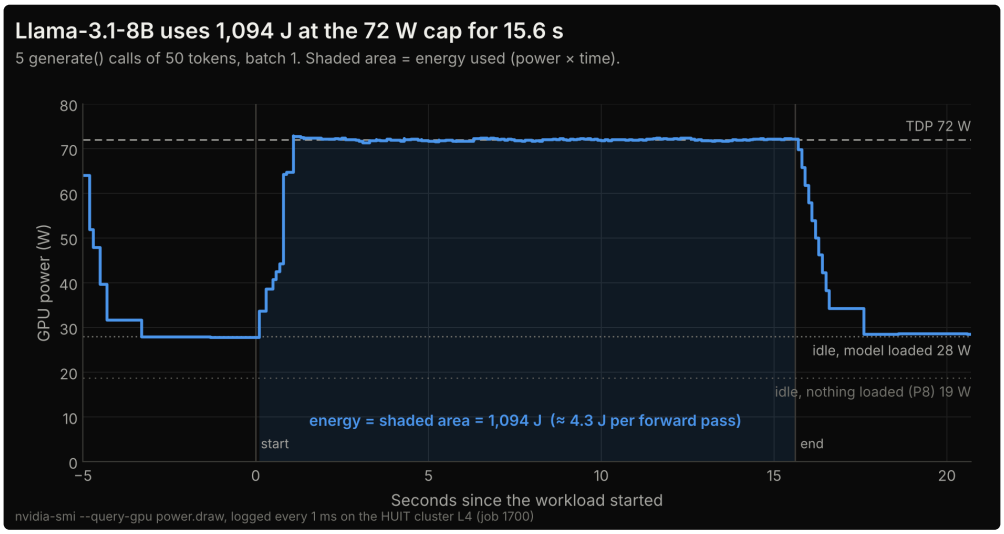
The trace reads ~28 W before the run, then a ~1 s ramp as clocks come up. The GPU sits on the 72 W cap until the work ends, then decays back to ~28 W over ~2 s.

2 def of idle. The plain nvidia-smi in Part III showed 16 to 19 W. With a model loaded and the GPU recently active, it settles at ~28 W instead and doesn't drop to P8 within the 5 s padding. The plots show both lines. For "energy above idle" I subtract the 28 W figure, since that's the real floor the workload sits on.

Exercise III: Power and Energy for Llama-3.1-8B

Q11. Record the power consumption of a forward pass of Llama 3.1-8B. Plot power over time, with idle power and the hardware TDP as dashed horizontal lines.

Llama-3.1-8B used 1,094 J over 15.6 s, about 4.3 J per forward pass, drawing ~70 W, essentially the full 72 W cap.



Q12. Calculate the energy usage for the workload's execution.

Energy is the sum of each power reading times the time it covered, i.e the area under the power curve between the start and end lines:

$$E = \sum P_i \cdot \Delta t_i$$
$$\approx 70.1\ \text{W} \times 15.61\ \text{s}$$
$$= 1{,}094\ \text{J}$$

Model	Time (s)	Avg power (W)	Idle (W)	Energy (J)	Energy above idle (J)	Per generate() (J)	Per token (J)
Llama-3.1-8B, batch 1	15.61	70.1	28.6	**1,094**	648	218.8	4.38

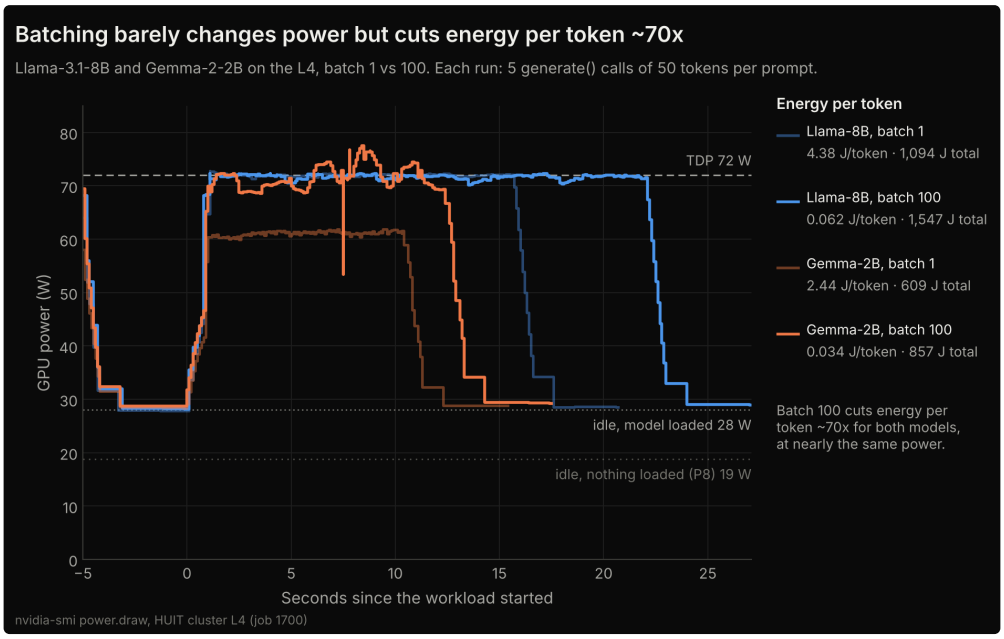
Per forward pass: each generate() is 1 prefill pass plus 50 decode passes, so 1 forward pass costs about 218.8 / 51 ≈ **4.3 J**.

Why: 5 × 50 = 250 tokens in 15.6 s is 62 ms per token. Each decode step reads all 8B fp16 weights (~16 GB), so the GPU streams about 16 GB / 62 ms ≈ **257 GB/s, or 86% of the L4's bandwidth**. This is GEMM B from Exercise II at the scale of a whole model: batch-1 decode is memory-bound. It still draws the full 72 W because DRAM running at full bandwidth is itself a big power draw.

Exercise IV: Comparing Power Profiles Across Workloads

Q13. Vary batch size between 1 and 100, for Llama-3.1-8B and Gemma-2-2B. Plot all 4 power curves on one graph and comment on how the power profile and energy consumption compare.

Going from batch 1 to batch 100 barely changes power but cuts energy per token about 70x, for both Llama-3.1-8B and Gemma-2-2B.



Model	Batch	Time (s)	Avg power (W)	Peak (W)	Energy (J)	Per token (J)	Tokens/s
Llama-3.1-8B	1	15.61	70.1	72.8	1,094	**4.38**	16
Llama-3.1-8B	100	21.98	70.4	72.3	1,547	**0.062**	1,137
Gemma-2-2B	1	10.36	58.8	61.9	609	**2.44**	24
Gemma-2-2B	100	12.42	69.1	77.5	857	**0.034**	2,013

Tokens/s = 5 iterations × 50 tokens × batch / time.

Batch size. Batch 100 is 100x the tokens but only 1.4x the time for Llama and 1.2x for Gemma. Power stays near the cap either way, so total energy rises only 1.4x while energy per token drops 71x (Llama) and 72x (Gemma). The reason is the same as GEMM B: at batch 1, each weight read from DRAM serves 1 token. At batch 100 the same read serves 100 tokens, so the extra work is nearly free until compute becomes the limit.

Gemma-2-2B at batch 1 doesn't reach the power cap. It sits at ~61 W while every other run pins 72 W. At 41 ms per token it moves only ~5.2 GB of weights per step, about 127 GB/s, so it isn't bandwidth-bound either. My read is that it's overhead-bound: Gemma 2's architecture (logit softcapping, alternating sliding-window attention) launches more small kernels per token, and at batch 1 the GPU idles between them. I didn't profile it, so this is a hypothesis. A PyTorch Profiler trace of 1 decode step would confirm it.

Gemma at batch 100 briefly exceeds TDP (77.5 W peak). The power limiter works on an averaged window, so short spikes above 72 W are allowed.

Llama-8B vs Gemma-2B. Gemma is ~3x smaller, but per token it costs only 44% less at batch 1 and 45% less at batch 100. At batch 1 Gemma isn't using the memory system efficiently. At batch 100 both runs are pinned at the power cap, so the gap shows up as time rather than watts.

Takeaways

- The L4 is a low-bandwidth, power-capped part. Its ridge point is 403 FLOP/B, higher than an H100's, and any sustained workload pins it at 72 W.
- Decode is memory-bound and prefill is compute-bound. GEMM B (M = 1) hits 81% of DRAM bandwidth, and full-model decode for Llama-8B hits 86%.
- Batching is the biggest energy lever here: 100x batch cut energy per token about 70x at nearly constant power.
- Idle power on the L4 measured 16 to 19 W with nothing loaded (P8), but ~28 W with a model still loaded right after work, since the live CUDA context keeps the GPU out of P8. Which 1 counts as idle? More of a question.

Definitions

Term	Meaning
AI (arithmetic intensity)	FLOPs per byte moved from memory. High AI = lots of reuse per byte
Bandwidth	Rate data moves between GPU memory and the execution units (GB/s)
Batch size	Number of independent prompts processed together
Compute-bound	Speed is limited by the execution units (right of the ridge)
Decode	Generating output tokens 1 at a time. Memory-bound at small batch
DRAM	The GPU's main memory (24 GB on the L4)
Energy (J)	Power × time. What you pay for
FLOP	1 floating-point add or multiply
fp16	16-bit floating point numbers, 2 bytes each
GEMM	General matrix multiply, C = A × B
Kernel	A single function launched on the GPU
Memory-bound	Speed is limited by memory bandwidth (left of the ridge)
nvidia-smi	NVIDIA's command-line GPU monitoring tool

nvidia-smi	NVIDIA's command-line GPU monitoring tool
Parameters	The learned numbers (weights) that make up a model. 8B = 8 billion
P-state	GPU power mode. P0 = full speed, P8 = deep idle
Power (W)	Instantaneous rate of energy use
Prefill	Processing the whole prompt in 1 pass. Compute-bound
Ridge point	AI where a roofline turns flat. Peak FLOP/s / bandwidth
Roofline	Plot of attainable FLOP/s vs AI for a given chip
Slurm, srun, sbatch	The cluster's job scheduler and its interactive / batch job commands
Sparsity	Skipping zero weights in a fixed pattern. Doubles NVIDIA's quoted peaks, but doesn't apply to dense matrices
Tensor Cores	GPU units specialized for small matrix multiplies. The 121 TFLOP/s fp16 peak comes from these
TDP / power limit	Max power the GPU may draw. The L4 throttles its clocks to stay under 72 W
Token	A word or word-piece. LLMs generate 1 token per decode step

Sources

- NVIDIA L4 Tensor Core GPU spec page
- Lab 3 handout
- Code and raw results on GitHub